Flash / Metadata逐byte位址與CAN封包移植對照

1. 適用角色與閱讀方式

本節對照 **F13 (MCU CAN device)** 的CAN3映像與 **F24 (MCU CAN host)** 的BUR1映像。 後續F24 (MCU CAN device) 是另一組Boot/App，不能直接使用本表F24 host的分區或BUR1更新協定。 CAN3 / BUR1的metadata位置相對App相同，但前96 bytes欄位排列不同；0x60..0x7F才是共用UPD1。 這是Flash / 封包與原始碼閱讀指南，不是跳過驗證或直接用CAN analyzer擦寫的操作指令。

Flash絕對位址、BIN offset與metadata offset

```
metadata_base = app_start + 0x100
field_flash_address = app_start + 0x100 + metadata_offset
field_bin_offset = 0x100 + metadata_offset
crc_flash_address = app_start + image_size - 4
crc_bin_offset = image_size - 4
```

例如F13的target_id在metadata offset0x0C..0x0D：Flash為0x0210C..0x0210D， BIN為0x10C..0x10D。F24 host的app_start在metadata offset0x18..0x1B：Flash為0x05118..0x0511B。 這兩個offset不能跨CAN3 / BUR1直接互換；所有多byte整數皆little-endian，低位byte在低位址。

區域	F13 (MCU CAN device)	F24 (MCU CAN host)
Code Flash	0x00000..0x1FFFF	0x00000..0x3FFFF
Boot（含低位向量 / option等受保護區）	0x00000..0x01FFF	0x00000..0x04FFF
App可用範圍	0x02000..0x1FFFF	0x05000..0x3FBFF
App固定跳轉表	0x02000..0x020FF	0x05000..0x050FF
Metadata整區	0x02100..0x021FF	0x05100..0x051FF
已定義core / 含UPD1	0x02100..0x0217F	0x05100..0x0517F
Reserved，全部0xFF	0x02180..0x021FF	0x05180..0x051FF
Linked code起點	0x02200	0x05200
最後保留區	無另設此表所述尾端保留區；仍須保留既定debug trace RAM	0x3FC00..0x3FFFF，不擦寫
Image CRC32	實際BIN最後4 bytes，不是固定Flash尾端	同左

不同CRC不要混淆

名稱	存放 / 傳送位置	計算範圍
Image CRC32	BIN最後4 bytes / LE32	BIN[0..image_size-5]，含跳轉表、metadata、reserved及填洞FF；排除CRC自身
Metadata staging CRC32	CAN SEAL或Host BEGIN的command payload	完整metadata 256 bytes；不是多佔用一組metadata內的CRC欄位
CAN block CRC32	BLOCK_BEGIN及BLOCK_COMMIT的Byte4..7	本區塊實際image bytes，最多64；不含control header或未傳送padding
Custom Transfer CRC8	固定8-byte payload的Byte7	該Transfer封包Byte0..6；不是CAN Upgrade的固定Byte7規則
CAN控制器鏈路CRC	CAN frame硬體協定欄位	由CAN控制器處理，不佔本文件Byte0..7的payload
Host Link CRC16	UART frame的header CRC / payload CRC	CRC16/CCITT-FALSE；header覆蓋Byte2..9，payload CRC只覆蓋UART payload，不是image CRC
Intel HEX / S-record checksum	HEX / MOT每筆文字record	燒錄檔record完整性；不是App image CRC32

Image / metadata / block採CRC32/ISO-HDLC，reflected polynomial0xEDB88320，init及xorout0xFFFFFFFF。 CRC提供完整性，不是簽章、認證或防回滾。image_size隨linker結果改變，所以CRC實際address必須重新計算。

2. 每個metadata byte：address、範例值與來源

下表所有offset均相對metadata_base；address / offset以0x標示，value欄是兩位十六進位byte，bytes欄長度為十進位。「產生端來源」對照下一節可點選的真正範例程式；欄位後[0] / [1]等表示該欄位內的byte。 CAN3來源主要是ftable.inc / ftable_RL78_F13.asm；BUR1來源是package_f24_bridge.ps1。 UPD1來源是F13組語的policy words、F24 packager及共用image_policy.h。

F13 (MCU CAN device)：CAN3 欄位與產生端

Metadata offset	bytes	欄位	產生端來源符號 / 位置
0x00..0x03	4	CAN3 magic	IMAGE_MANIFEST_MAGIC
0x04..0x04	1	image_format	IMAGE_MANIFEST_FORMAT_INFO
0x05..0x05	1	core_length	IMAGE_MANIFEST_FORMAT_INFO
0x06..0x07	2	flags	IMAGE_MANIFEST_FORMAT_INFO
0x08..0x09	2	protocol_id	IMAGE_MANIFEST_PROTOCOL
0x0A..0x0A	1	protocol_version	IMAGE_MANIFEST_PROTOCOL
0x0B..0x0B	1	checksum_type	IMAGE_MANIFEST_PROTOCOL
0x0C..0x0D	2	target_id	IMAGE_MANIFEST_TARGET
0x0E..0x0E	1	can_mode_mask	IMAGE_MANIFEST_TARGET
0x0F..0x0F	1	can_id_format	IMAGE_MANIFEST_TARGET
0x10..0x13	4	app_version	APP_IMAGE_VERSION
0x14..0x17	4	app_build	APP_IMAGE_BUILD
0x18..0x1B	4	image_size	IMAGE_SIZE_POST_BUILD / post-build
0x1C..0x1F	4	app_start	FLASH_TABLE
0x20..0x23	4	app_capacity	APP_REGION_SIZE
0x24..0x27	4	flash_size	CODE_FLASH_SIZE
0x28..0x2B	4	boot_size	BOOT_RESERVED_SIZE
0x2C..0x2F	4	erase_size	FLASH_ERASE_BLOCK_SIZE
0x30..0x33	4	nominal_rate_mask	CAN_NOMINAL_BITRATE_MASK
0x34..0x37	4	data_rate_mask	CAN_DATA_BITRATE_MASK
0x38..0x3B	4	request_id	CAN_REQUEST_ID
0x3C..0x3F	4	response_id	CAN_RESPONSE_ID
0x40..0x4F	16	mcu_part ASCII	MCU_PART_0..1 / zero padding
0x50..0x5B	12	mcu_family ASCII	MCU_FAMILY_0..1 / zero padding

Metadata offset	bytes	欄位	產生端來源符號 / 位置
0x5C..0x5F	4	minimum_size	MINIMUM_IMAGE_SIZE
0x60..0x63	4	UPD1 magic	UPD1 policy
0x64..0x64	1	policy_version	UPD1 policy
0x65..0x65	1	policy_size	UPD1 policy
0x66..0x66	1	image_role	UPD1 policy
0x67..0x67	1	update_transports	UPD1 policy
0x68..0x6B	4	irq_abi	UPD1 policy
0x6C..0x6F	4	minimum_boot	UPD1 policy
0x70..0x73	4	device_uart_baud	UPD1 policy
0x74..0x7F	12	reserved_zero	UPD1 policy
0x80..0xFF	128	reserved_FF	reserved region

F24 (MCU CAN host)：BUR1 欄位與產生端

Metadata offset	bytes	欄位	產生端來源符號 / 位置
0x00..0x03	4	BUR1 magic	Put32 0
0x04..0x04	1	contract_version	image[0x104]
0x05..0x05	1	core_length	image[0x105]
0x06..0x06	1	image_role	image[0x106]
0x07..0x07	1	checksum_type	image[0x107]
0x08..0x09	2	board_id	BOARD_ID / image[0x108]
0x0A..0x0B	2	contract_id	image[0x10A] / zero padding
0x0C..0x0F	4	image_size	Put32 12 / linker-derived size
0x10..0x13	4	app_version	app_image.json / Put32 16
0x14..0x17	4	app_build	app_image.json / Put32 20
0x18..0x1B	4	app_start	APP_START / Put32 24
0x1C..0x1F	4	app_capacity	APP_LIMIT - APP_START / Put32 28
0x20..0x23	4	flash_size	FLASH_SIZE / Put32 32
0x24..0x27	4	boot_size	APP_START / Put32 36
0x28..0x2B	4	erase_size	ERASE_SIZE / Put32 40
0x2C..0x2F	4	minimum_size	MINIMUM_SIZE / Put32 44
0x30..0x33	4	boot_uart_baud	UART_BAUD / Put32 48
0x34..0x3F	12	mcu_family ASCII	ASCII CopyTo image[0x134]
0x40..0x4F	16	mcu_part ASCII	ASCII CopyTo image[0x140]
0x50..0x53	4	minimum_boot_copy	BOOT_VERSION / Put32 80
0x54..0x5F	12	reserved_zero	core initialized zero
0x60..0x63	4	UPD1 magic	UPD1 policy
0x64..0x64	1	policy_version	UPD1 policy
0x65..0x65	1	policy_size	UPD1 policy

Metadata offset	bytes	欄位	產生端來源符號 / 位置
0x66..0x66	1	image_role	UPD1 policy
0x67..0x67	1	update_transports	UPD1 policy
0x68..0x6B	4	irq_abi	UPD1 policy
0x6C..0x6F	4	minimum_boot	UPD1 policy
0x70..0x73	4	device_uart_baud	UPD1 policy
0x74..0x7F	12	reserved_zero	UPD1 policy
0x80..0xFF	128	reserved_FF	reserved region

逐 byte 實體位址與範例值

方括號是欄位內byte索引；LE數值的[0]是最低有效byte。ASCII依字元順序，reserved依原始順序。 範例值取配套BIN，不是所有專案的固定值；每個metadata byte皆各列一行，包含128個reserved FF。

Metadata 0x00..0x1F

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0x00	0x02100	43	CAN3 magic[0]	0x05100	42	BUR1 magic[0]
0x01	0x02101	41	CAN3 magic[1]	0x05101	55	BUR1 magic[1]
0x02	0x02102	4E	CAN3 magic[2]	0x05102	52	BUR1 magic[2]
0x03	0x02103	33	CAN3 magic[3]	0x05103	31	BUR1 magic[3]
0x04	0x02104	06	image_format[0]	0x05104	03	contract_version[0]
0x05	0x02105	80	core_length[0]	0x05105	80	core_length[0]
0x06	0x02106	03	flags[0]	0x05106	02	image_role[0]
0x07	0x02107	00	flags[1]	0x05107	01	checksum_type[0]
0x08	0x02108	01	protocol_id[0]	0x05108	01	board_id[0]
0x09	0x02109	00	protocol_id[1]	0x05109	01	board_id[1]
0x0A	0x0210A	05	protocol_version[0]	0x0510A	02	contract_id[0]
0x0B	0x0210B	01	checksum_type[0]	0x0510B	00	contract_id[1]
0x0C	0x0210C	3B	target_id[0]	0x0510C	C8	image_size[0]
0x0D	0x0210D	F1	target_id[1]	0x0510D	A6	image_size[1]
0x0E	0x0210E	01	can_mode_mask[0]	0x0510E	00	image_size[2]
0x0F	0x0210F	00	can_id_format[0]	0x0510F	00	image_size[3]
0x10	0x02110	00	app_version[0]	0x05110	00	app_version[0]
0x11	0x02111	05	app_version[1]	0x05111	05	app_version[1]
0x12	0x02112	01	app_version[2]	0x05112	01	app_version[2]
0x13	0x02113	00	app_version[3]	0x05113	00	app_version[3]
0x14	0x02114	97	app_build[0]	0x05114	97	app_build[0]
0x15	0x02115	28	app_build[1]	0x05115	28	app_build[1]
0x16	0x02116	35	app_build[2]	0x05116	35	app_build[2]
0x17	0x02117	01	app_build[3]	0x05117	01	app_build[3]

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0x18	0x02118	2C	image_size[0]	0x05118	00	app_start[0]
0x19	0x02119	4B	image_size[1]	0x05119	50	app_start[1]
0x1A	0x0211A	00	image_size[2]	0x0511A	00	app_start[2]
0x1B	0x0211B	00	image_size[3]	0x0511B	00	app_start[3]
0x1C	0x0211C	00	app_start[0]	0x0511C	00	app_capacity[0]
0x1D	0x0211D	20	app_start[1]	0x0511D	AC	app_capacity[1]
0x1E	0x0211E	00	app_start[2]	0x0511E	03	app_capacity[2]
0x1F	0x0211F	00	app_start[3]	0x0511F	00	app_capacity[3]

Metadata 0x20..0x3F

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0x20	0x02120	00	app_capacity[0]	0x05120	00	flash_size[0]
0x21	0x02121	E0	app_capacity[1]	0x05121	00	flash_size[1]
0x22	0x02122	01	app_capacity[2]	0x05122	04	flash_size[2]
0x23	0x02123	00	app_capacity[3]	0x05123	00	flash_size[3]
0x24	0x02124	00	flash_size[0]	0x05124	00	boot_size[0]
0x25	0x02125	00	flash_size[1]	0x05125	50	boot_size[1]
0x26	0x02126	02	flash_size[2]	0x05126	00	boot_size[2]
0x27	0x02127	00	flash_size[3]	0x05127	00	boot_size[3]
0x28	0x02128	00	boot_size[0]	0x05128	00	erase_size[0]
0x29	0x02129	20	boot_size[1]	0x05129	04	erase_size[1]
0x2A	0x0212A	00	boot_size[2]	0x0512A	00	erase_size[2]
0x2B	0x0212B	00	boot_size[3]	0x0512B	00	erase_size[3]
0x2C	0x0212C	00	erase_size[0]	0x0512C	04	minimum_size[0]
0x2D	0x0212D	04	erase_size[1]	0x0512D	02	minimum_size[1]
0x2E	0x0212E	00	erase_size[2]	0x0512E	00	minimum_size[2]
0x2F	0x0212F	00	erase_size[3]	0x0512F	00	minimum_size[3]
0x30	0x02130	04	nominal_rate_mask[0]	0x05130	00	boot_uart_baud[0]
0x31	0x02131	00	nominal_rate_mask[1]	0x05131	C2	boot_uart_baud[1]
0x32	0x02132	00	nominal_rate_mask[2]	0x05132	01	boot_uart_baud[2]
0x33	0x02133	00	nominal_rate_mask[3]	0x05133	00	boot_uart_baud[3]
0x34	0x02134	00	data_rate_mask[0]	0x05134	52	mcu_family ASCII[0]
0x35	0x02135	00	data_rate_mask[1]	0x05135	4C	mcu_family ASCII[1]
0x36	0x02136	00	data_rate_mask[2]	0x05136	37	mcu_family ASCII[2]
0x37	0x02137	00	data_rate_mask[3]	0x05137	38	mcu_family ASCII[3]

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0x38	0x02138	00	request_id[0]	0x05138	2F	mcu_family ASCII[4]
0x39	0x02139	07	request_id[1]	0x05139	46	mcu_family ASCII[5]
0x3A	0x0213A	00	request_id[2]	0x0513A	32	mcu_family ASCII[6]
0x3B	0x0213B	00	request_id[3]	0x0513B	34	mcu_family ASCII[7]
0x3C	0x0213C	01	response_id[0]	0x0513C	00	mcu_family ASCII[8]
0x3D	0x0213D	07	response_id[1]	0x0513D	00	mcu_family ASCII[9]
0x3E	0x0213E	00	response_id[2]	0x0513E	00	mcu_family ASCII[10]
0x3F	0x0213F	00	response_id[3]	0x0513F	00	mcu_family ASCII[11]

Metadata 0x40..0x5F

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0x40	0x02140	52	mcu_part ASCII[0]	0x05140	52	mcu_part ASCII[0]
0x41	0x02141	35	mcu_part ASCII[1]	0x05141	37	mcu_part ASCII[1]
0x42	0x02142	46	mcu_part ASCII[2]	0x05142	46	mcu_part ASCII[2]
0x43	0x02143	31	mcu_part ASCII[3]	0x05143	31	mcu_part ASCII[3]
0x44	0x02144	30	mcu_part ASCII[4]	0x05144	32	mcu_part ASCII[4]
0x45	0x02145	42	mcu_part ASCII[5]	0x05145	34	mcu_part ASCII[5]
0x46	0x02146	4D	mcu_part ASCII[6]	0x05146	46	mcu_part ASCII[6]
0x47	0x02147	47	mcu_part ASCII[7]	0x05147	4C	mcu_part ASCII[7]
0x48	0x02148	00	mcu_part ASCII[8]	0x05148	4A	mcu_part ASCII[8]
0x49	0x02149	00	mcu_part ASCII[9]	0x05149	33	mcu_part ASCII[9]
0x4A	0x0214A	00	mcu_part ASCII[10]	0x0514A	78	mcu_part ASCII[10]
0x4B	0x0214B	00	mcu_part ASCII[11]	0x0514B	46	mcu_part ASCII[11]
0x4C	0x0214C	00	mcu_part ASCII[12]	0x0514C	42	mcu_part ASCII[12]
0x4D	0x0214D	00	mcu_part ASCII[13]	0x0514D	00	mcu_part ASCII[13]
0x4E	0x0214E	00	mcu_part ASCII[14]	0x0514E	00	mcu_part ASCII[14]
0x4F	0x0214F	00	mcu_part ASCII[15]	0x0514F	00	mcu_part ASCII[15]
0x50	0x02150	52	mcu_family ASCII[0]	0x05150	00	minimum_boot_copy[0]
0x51	0x02151	4C	mcu_family ASCII[1]	0x05151	02	minimum_boot_copy[1]
0x52	0x02152	37	mcu_family ASCII[2]	0x05152	01	minimum_boot_copy[2]
0x53	0x02153	38	mcu_family ASCII[3]	0x05153	00	minimum_boot_copy[3]
0x54	0x02154	2F	mcu_family ASCII[4]	0x05154	00	reserved_zero[0]
0x55	0x02155	46	mcu_family ASCII[5]	0x05155	00	reserved_zero[1]
0x56	0x02156	31	mcu_family ASCII[6]	0x05156	00	reserved_zero[2]
0x57	0x02157	33	mcu_family ASCII[7]	0x05157	00	reserved_zero[3]

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0x58	0x02158	00	mcu_family ASCII[8]	0x05158	00	reserved_zero[4]
0x59	0x02159	00	mcu_family ASCII[9]	0x05159	00	reserved_zero[5]
0x5A	0x0215A	00	mcu_family ASCII[10]	0x0515A	00	reserved_zero[6]
0x5B	0x0215B	00	mcu_family ASCII[11]	0x0515B	00	reserved_zero[7]
0x5C	0x0215C	04	minimum_size[0]	0x0515C	00	reserved_zero[8]
0x5D	0x0215D	02	minimum_size[1]	0x0515D	00	reserved_zero[9]
0x5E	0x0215E	00	minimum_size[2]	0x0515E	00	reserved_zero[10]
0x5F	0x0215F	00	minimum_size[3]	0x0515F	00	reserved_zero[11]

Metadata 0x60..0x7F

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0x60	0x02160	55	UPD1 magic[0]	0x05160	55	UPD1 magic[0]
0x61	0x02161	50	UPD1 magic[1]	0x05161	50	UPD1 magic[1]
0x62	0x02162	44	UPD1 magic[2]	0x05162	44	UPD1 magic[2]
0x63	0x02163	31	UPD1 magic[3]	0x05163	31	UPD1 magic[3]
0x64	0x02164	01	policy_version[0]	0x05164	01	policy_version[0]
0x65	0x02165	20	policy_size[0]	0x05165	20	policy_size[0]
0x66	0x02166	01	image_role[0]	0x05166	02	image_role[0]
0x67	0x02167	01	update_transports[0]	0x05167	04	update_transports[0]
0x68	0x02168	46	irq_abi[0]	0x05168	46	irq_abi[0]
0x69	0x02169	31	irq_abi[1]	0x05169	32	irq_abi[1]
0x6A	0x0216A	56	irq_abi[2]	0x0516A	56	irq_abi[2]
0x6B	0x0216B	31	irq_abi[3]	0x0516B	31	irq_abi[3]
0x6C	0x0216C	00	minimum_boot[0]	0x0516C	00	minimum_boot[0]
0x6D	0x0216D	05	minimum_boot[1]	0x0516D	02	minimum_boot[1]
0x6E	0x0216E	01	minimum_boot[2]	0x0516E	01	minimum_boot[2]
0x6F	0x0216F	00	minimum_boot[3]	0x0516F	00	minimum_boot[3]
0x70	0x02170	00	device_uart_baud[0]	0x05170	00	device_uart_baud[0]
0x71	0x02171	00	device_uart_baud[1]	0x05171	00	device_uart_baud[1]
0x72	0x02172	00	device_uart_baud[2]	0x05172	00	device_uart_baud[2]
0x73	0x02173	00	device_uart_baud[3]	0x05173	00	device_uart_baud[3]
0x74	0x02174	00	reserved_zero[0]	0x05174	00	reserved_zero[0]
0x75	0x02175	00	reserved_zero[1]	0x05175	00	reserved_zero[1]
0x76	0x02176	00	reserved_zero[2]	0x05176	00	reserved_zero[2]
0x77	0x02177	00	reserved_zero[3]	0x05177	00	reserved_zero[3]

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0x78	0x02178	00	reserved_zero[4]	0x05178	00	reserved_zero[4]
0x79	0x02179	00	reserved_zero[5]	0x05179	00	reserved_zero[5]
0x7A	0x0217A	00	reserved_zero[6]	0x0517A	00	reserved_zero[6]
0x7B	0x0217B	00	reserved_zero[7]	0x0517B	00	reserved_zero[7]
0x7C	0x0217C	00	reserved_zero[8]	0x0517C	00	reserved_zero[8]
0x7D	0x0217D	00	reserved_zero[9]	0x0517D	00	reserved_zero[9]
0x7E	0x0217E	00	reserved_zero[10]	0x0517E	00	reserved_zero[10]
0x7F	0x0217F	00	reserved_zero[11]	0x0517F	00	reserved_zero[11]

Metadata 0x80..0x9F

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0x80	0x02180	FF	reserved_FF[0]	0x05180	FF	reserved_FF[0]
0x81	0x02181	FF	reserved_FF[1]	0x05181	FF	reserved_FF[1]
0x82	0x02182	FF	reserved_FF[2]	0x05182	FF	reserved_FF[2]
0x83	0x02183	FF	reserved_FF[3]	0x05183	FF	reserved_FF[3]
0x84	0x02184	FF	reserved_FF[4]	0x05184	FF	reserved_FF[4]
0x85	0x02185	FF	reserved_FF[5]	0x05185	FF	reserved_FF[5]
0x86	0x02186	FF	reserved_FF[6]	0x05186	FF	reserved_FF[6]
0x87	0x02187	FF	reserved_FF[7]	0x05187	FF	reserved_FF[7]
0x88	0x02188	FF	reserved_FF[8]	0x05188	FF	reserved_FF[8]
0x89	0x02189	FF	reserved_FF[9]	0x05189	FF	reserved_FF[9]
0x8A	0x0218A	FF	reserved_FF[10]	0x0518A	FF	reserved_FF[10]
0x8B	0x0218B	FF	reserved_FF[11]	0x0518B	FF	reserved_FF[11]
0x8C	0x0218C	FF	reserved_FF[12]	0x0518C	FF	reserved_FF[12]
0x8D	0x0218D	FF	reserved_FF[13]	0x0518D	FF	reserved_FF[13]
0x8E	0x0218E	FF	reserved_FF[14]	0x0518E	FF	reserved_FF[14]
0x8F	0x0218F	FF	reserved_FF[15]	0x0518F	FF	reserved_FF[15]
0x90	0x02190	FF	reserved_FF[16]	0x05190	FF	reserved_FF[16]
0x91	0x02191	FF	reserved_FF[17]	0x05191	FF	reserved_FF[17]
0x92	0x02192	FF	reserved_FF[18]	0x05192	FF	reserved_FF[18]
0x93	0x02193	FF	reserved_FF[19]	0x05193	FF	reserved_FF[19]
0x94	0x02194	FF	reserved_FF[20]	0x05194	FF	reserved_FF[20]
0x95	0x02195	FF	reserved_FF[21]	0x05195	FF	reserved_FF[21]
0x96	0x02196	FF	reserved_FF[22]	0x05196	FF	reserved_FF[22]
0x97	0x02197	FF	reserved_FF[23]	0x05197	FF	reserved_FF[23]

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0x98	0x02198	FF	reserved_FF[24]	0x05198	FF	reserved_FF[24]
0x99	0x02199	FF	reserved_FF[25]	0x05199	FF	reserved_FF[25]
0x9A	0x0219A	FF	reserved_FF[26]	0x0519A	FF	reserved_FF[26]
0x9B	0x0219B	FF	reserved_FF[27]	0x0519B	FF	reserved_FF[27]
0x9C	0x0219C	FF	reserved_FF[28]	0x0519C	FF	reserved_FF[28]
0x9D	0x0219D	FF	reserved_FF[29]	0x0519D	FF	reserved_FF[29]
0x9E	0x0219E	FF	reserved_FF[30]	0x0519E	FF	reserved_FF[30]
0x9F	0x0219F	FF	reserved_FF[31]	0x0519F	FF	reserved_FF[31]

Metadata 0xA0..0xBF

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0xA0	0x021A0	FF	reserved_FF[32]	0x051A0	FF	reserved_FF[32]
0xA1	0x021A1	FF	reserved_FF[33]	0x051A1	FF	reserved_FF[33]
0xA2	0x021A2	FF	reserved_FF[34]	0x051A2	FF	reserved_FF[34]
0xA3	0x021A3	FF	reserved_FF[35]	0x051A3	FF	reserved_FF[35]
0xA4	0x021A4	FF	reserved_FF[36]	0x051A4	FF	reserved_FF[36]
0xA5	0x021A5	FF	reserved_FF[37]	0x051A5	FF	reserved_FF[37]
0xA6	0x021A6	FF	reserved_FF[38]	0x051A6	FF	reserved_FF[38]
0xA7	0x021A7	FF	reserved_FF[39]	0x051A7	FF	reserved_FF[39]
0xA8	0x021A8	FF	reserved_FF[40]	0x051A8	FF	reserved_FF[40]
0xA9	0x021A9	FF	reserved_FF[41]	0x051A9	FF	reserved_FF[41]
0xAA	0x021AA	FF	reserved_FF[42]	0x051AA	FF	reserved_FF[42]
0xAB	0x021AB	FF	reserved_FF[43]	0x051AB	FF	reserved_FF[43]
0xAC	0x021AC	FF	reserved_FF[44]	0x051AC	FF	reserved_FF[44]
0xAD	0x021AD	FF	reserved_FF[45]	0x051AD	FF	reserved_FF[45]
0xAE	0x021AE	FF	reserved_FF[46]	0x051AE	FF	reserved_FF[46]
0xAF	0x021AF	FF	reserved_FF[47]	0x051AF	FF	reserved_FF[47]
0xB0	0x021B0	FF	reserved_FF[48]	0x051B0	FF	reserved_FF[48]
0xB1	0x021B1	FF	reserved_FF[49]	0x051B1	FF	reserved_FF[49]
0xB2	0x021B2	FF	reserved_FF[50]	0x051B2	FF	reserved_FF[50]
0xB3	0x021B3	FF	reserved_FF[51]	0x051B3	FF	reserved_FF[51]
0xB4	0x021B4	FF	reserved_FF[52]	0x051B4	FF	reserved_FF[52]
0xB5	0x021B5	FF	reserved_FF[53]	0x051B5	FF	reserved_FF[53]
0xB6	0x021B6	FF	reserved_FF[54]	0x051B6	FF	reserved_FF[54]
0xB7	0x021B7	FF	reserved_FF[55]	0x051B7	FF	reserved_FF[55]

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0xB8	0x021B8	FF	reserved_FF[56]	0x051B8	FF	reserved_FF[56]
0xB9	0x021B9	FF	reserved_FF[57]	0x051B9	FF	reserved_FF[57]
0xBA	0x021BA	FF	reserved_FF[58]	0x051BA	FF	reserved_FF[58]
0xBB	0x021BB	FF	reserved_FF[59]	0x051BB	FF	reserved_FF[59]
0xBC	0x021BC	FF	reserved_FF[60]	0x051BC	FF	reserved_FF[60]
0xBD	0x021BD	FF	reserved_FF[61]	0x051BD	FF	reserved_FF[61]
0xBE	0x021BE	FF	reserved_FF[62]	0x051BE	FF	reserved_FF[62]
0xBF	0x021BF	FF	reserved_FF[63]	0x051BF	FF	reserved_FF[63]

Metadata 0xC0..0xDF

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0xC0	0x021C0	FF	reserved_FF[64]	0x051C0	FF	reserved_FF[64]
0xC1	0x021C1	FF	reserved_FF[65]	0x051C1	FF	reserved_FF[65]
0xC2	0x021C2	FF	reserved_FF[66]	0x051C2	FF	reserved_FF[66]
0xC3	0x021C3	FF	reserved_FF[67]	0x051C3	FF	reserved_FF[67]
0xC4	0x021C4	FF	reserved_FF[68]	0x051C4	FF	reserved_FF[68]
0xC5	0x021C5	FF	reserved_FF[69]	0x051C5	FF	reserved_FF[69]
0xC6	0x021C6	FF	reserved_FF[70]	0x051C6	FF	reserved_FF[70]
0xC7	0x021C7	FF	reserved_FF[71]	0x051C7	FF	reserved_FF[71]
0xC8	0x021C8	FF	reserved_FF[72]	0x051C8	FF	reserved_FF[72]
0xC9	0x021C9	FF	reserved_FF[73]	0x051C9	FF	reserved_FF[73]
0xCA	0x021CA	FF	reserved_FF[74]	0x051CA	FF	reserved_FF[74]
0xCB	0x021CB	FF	reserved_FF[75]	0x051CB	FF	reserved_FF[75]
0xCC	0x021CC	FF	reserved_FF[76]	0x051CC	FF	reserved_FF[76]
0xCD	0x021CD	FF	reserved_FF[77]	0x051CD	FF	reserved_FF[77]
0xCE	0x021CE	FF	reserved_FF[78]	0x051CE	FF	reserved_FF[78]
0xCF	0x021CF	FF	reserved_FF[79]	0x051CF	FF	reserved_FF[79]
0xD0	0x021D0	FF	reserved_FF[80]	0x051D0	FF	reserved_FF[80]
0xD1	0x021D1	FF	reserved_FF[81]	0x051D1	FF	reserved_FF[81]
0xD2	0x021D2	FF	reserved_FF[82]	0x051D2	FF	reserved_FF[82]
0xD3	0x021D3	FF	reserved_FF[83]	0x051D3	FF	reserved_FF[83]
0xD4	0x021D4	FF	reserved_FF[84]	0x051D4	FF	reserved_FF[84]
0xD5	0x021D5	FF	reserved_FF[85]	0x051D5	FF	reserved_FF[85]
0xD6	0x021D6	FF	reserved_FF[86]	0x051D6	FF	reserved_FF[86]
0xD7	0x021D7	FF	reserved_FF[87]	0x051D7	FF	reserved_FF[87]

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0xD8	0x021D8	FF	reserved_FF[88]	0x051D8	FF	reserved_FF[88]
0xD9	0x021D9	FF	reserved_FF[89]	0x051D9	FF	reserved_FF[89]
0xDA	0x021DA	FF	reserved_FF[90]	0x051DA	FF	reserved_FF[90]
0xDB	0x021DB	FF	reserved_FF[91]	0x051DB	FF	reserved_FF[91]
0xDC	0x021DC	FF	reserved_FF[92]	0x051DC	FF	reserved_FF[92]
0xDD	0x021DD	FF	reserved_FF[93]	0x051DD	FF	reserved_FF[93]
0xDE	0x021DE	FF	reserved_FF[94]	0x051DE	FF	reserved_FF[94]
0xDF	0x021DF	FF	reserved_FF[95]	0x051DF	FF	reserved_FF[95]

Metadata 0xE0..0xFF

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0xE0	0x021E0	FF	reserved_FF[96]	0x051E0	FF	reserved_FF[96]
0xE1	0x021E1	FF	reserved_FF[97]	0x051E1	FF	reserved_FF[97]
0xE2	0x021E2	FF	reserved_FF[98]	0x051E2	FF	reserved_FF[98]
0xE3	0x021E3	FF	reserved_FF[99]	0x051E3	FF	reserved_FF[99]
0xE4	0x021E4	FF	reserved_FF[100]	0x051E4	FF	reserved_FF[100]
0xE5	0x021E5	FF	reserved_FF[101]	0x051E5	FF	reserved_FF[101]
0xE6	0x021E6	FF	reserved_FF[102]	0x051E6	FF	reserved_FF[102]
0xE7	0x021E7	FF	reserved_FF[103]	0x051E7	FF	reserved_FF[103]
0xE8	0x021E8	FF	reserved_FF[104]	0x051E8	FF	reserved_FF[104]
0xE9	0x021E9	FF	reserved_FF[105]	0x051E9	FF	reserved_FF[105]
0xEA	0x021EA	FF	reserved_FF[106]	0x051EA	FF	reserved_FF[106]
0xEB	0x021EB	FF	reserved_FF[107]	0x051EB	FF	reserved_FF[107]
0xEC	0x021EC	FF	reserved_FF[108]	0x051EC	FF	reserved_FF[108]
0xED	0x021ED	FF	reserved_FF[109]	0x051ED	FF	reserved_FF[109]
0xEE	0x021EE	FF	reserved_FF[110]	0x051EE	FF	reserved_FF[110]
0xEF	0x021EF	FF	reserved_FF[111]	0x051EF	FF	reserved_FF[111]
0xF0	0x021F0	FF	reserved_FF[112]	0x051F0	FF	reserved_FF[112]
0xF1	0x021F1	FF	reserved_FF[113]	0x051F1	FF	reserved_FF[113]
0xF2	0x021F2	FF	reserved_FF[114]	0x051F2	FF	reserved_FF[114]
0xF3	0x021F3	FF	reserved_FF[115]	0x051F3	FF	reserved_FF[115]
0xF4	0x021F4	FF	reserved_FF[116]	0x051F4	FF	reserved_FF[116]
0xF5	0x021F5	FF	reserved_FF[117]	0x051F5	FF	reserved_FF[117]
0xF6	0x021F6	FF	reserved_FF[118]	0x051F6	FF	reserved_FF[118]
0xF7	0x021F7	FF	reserved_FF[119]	0x051F7	FF	reserved_FF[119]

offset	F13 address	value	F13欄位[byte]	F24 host address	value	F24欄位[byte]
0xF8	0x021F8	FF	reserved_FF[120]	0x051F8	FF	reserved_FF[120]
0xF9	0x021F9	FF	reserved_FF[121]	0x051F9	FF	reserved_FF[121]
0xFA	0x021FA	FF	reserved_FF[122]	0x051FA	FF	reserved_FF[122]
0xFB	0x021FB	FF	reserved_FF[123]	0x051FB	FF	reserved_FF[123]
0xFC	0x021FC	FF	reserved_FF[124]	0x051FC	FF	reserved_FF[124]
0xFD	0x021FD	FF	reserved_FF[125]	0x051FD	FF	reserved_FF[125]
0xFE	0x021FE	FF	reserved_FF[126]	0x051FE	FF	reserved_FF[126]
0xFF	0x021FF	FF	reserved_FF[127]	0x051FF	FF	reserved_FF[127]

本表範例BIN與CRC位置

項目	F13 (MCU CAN device)	F24 (MCU CAN host)
image_size	0x4B2C	0xA6C8
stored image CRC32	0xCA0DF55C	0x864E811E
CRC32 Flash address	0x06B28..0x06B2B	0x0F6C4..0x0F6C7
metadata staging CRC32	0x15CBDE8C	0x7D759D88

F13 範例BIN SHA-256：`2C8C405A04A86F8C0B8E8CAC6AB834379BD43707C6600EA9F3666699ECED36B1`。 F24 host 範例BIN SHA-256：`B263AAC4300F94B5D385444FD4022D31CE3F8C2AE6E7FA5A0CC3095FEF84EC11`。

欄位的語意與修改限制

- core_length=128，不是整區256；0x80..0xFF目前必須全部FF，不能直接塞自訂欄位。
- flags=0x0003、protocol ID / version、checksum type及magic是契約，不是任意使用者標籤。 目前flags按完整0x0003比對，不能自行把其中bit當作新的可選功能開關。
- CAN mode mask的bit0代表Classical；目前F13 id_format=0代表standard。nominal_rate_mask的bit2表示500K，mask不是十進位500000。
- target_id / board_id是邏輯識別；MCU part / family是零結尾、其餘零填補ASCII，不是Flash address。
- image_size是實際封裝BIN含最後4-byte CRC的長度；app_capacity是最大容量，不能混用。
- app_version為0x00MMmmpp，app_build為32-bit建置識別；不是字串。image_size / CRC由post-build產生，勿手填規劃容量。
- UPD1 image_role：1=DEVICE_APP、2=HOST_BRIDGE_APP；update_transports：bit0 CAN、bit1 I2C、bit2 UART。

- irq_abi是固定向量介面的識別，minimum_boot是最低相容Boot版本，不是App最低版本。
- 現行CAN / HOST的policy_version=1，device_uart_baud及reserved_zero均為0。未來DEVICE UART的policy_version=2才解讀編譯時固定baud；不是上方PC□host的Host Baud。
- BUR1在offset0x50與0x6C重複minimum_boot，兩份必須相等；BUR1 header role與UPD1 role也須一致。

3. 對應的真正範例程式與修改位置

用途	原始碼 / 位置	要如何閱讀或修改
F13 metadata生成常數	ftable.inc	此檔現為生成檔；編輯App的app_image.json，由App-mode generator結合Boot contract產生常數；不可手改.inc
F13 byte排列 / policy	ftable_RL78_F13.asm	.appmeta從APP_MANIFEST_TABLE開始，.DB4依LE放4 bytes；尾128 bytes填FF
F13實際size / CRC封裝	generate_update_image.ps1	依map決定結尾、patch image_size、計算尾CRC；ExpectedTargetId等驗證亦須與新專案一致
F13 Boot驗證契約	can_boot_config.h / can_boot_metadata.inc	g_manifest_contract / MetadataValid核對欄位；不能只改App產生端
F13 CAN解碼與回覆	can_boot_transport.c / can_boot_block.inc	CAN_BootReceive、CAN_BootSend、BlockBegin / Segment / Check
F24 host分區	f24_bridge_layout.h	APP_START、APP_LIMIT、FLASH_SIZE、ERASE_SIZE、UART_BAUD等；不是F24 CAN device配置
F24 host App版本	app_image.json	每個App自己的version / build；不從其他App複製產表結果
F24 host metadata封裝	package_f24_bridge.ps1	Put32先加BIN offset0x100；產生BUR1、size、ASCII / UPD1及尾CRC
BUR1共用解析 / 驗證	bridge_update_contract.c	Bu_ParseManifest / Bu_ParseMetadataRegion；Bu_Read16 / Bu_Read32皆LE
共用政策常數	image_policy.h	ROLE / CAN / I2C / UART bits、IRQ ABI、core / policy長度
Tool image解析	firmware_update_profile.cpp / bridge_update_profile.cpp	檢查BIN、metadata、CRC及目的地；不是只改UI顯示字串
Tool CAN frame編碼	f13_boot_profile.cpp	BuildStartRequest、BuildBlockRequest、Client::StageMetadata / SendBlock，與下方逐byte表核對

F13組語常數與實際byte的關係

```
; Values from the paired F13 CAN example, not a new device configuration.
IMAGE_MANIFEST_FORMAT_INFO .EQU 0x00038006
IMAGE_MANIFEST_PROTOCOL    .EQU 0x01050001
IMAGE_MANIFEST_TARGET      .EQU 0x0001F13B
; .DB4 emits little-endian words:
; metadata 04..07 = 06 80 03 00
; metadata 08..0B = 01 00 05 01
; metadata 0C..0F = 3B F1 01 00
```

例如改target_id要核對F13 Boot的CAN_BOOT_TARGET_ID、App的IMAGE_MANIFEST_TARGET、 post-build驗證及Tool live INFO，不是只修改0x0210C。改CAN ID還須保持control / response與raw720..727不重疊。

C90示例：讀metadata而不依賴struct padding

```
/* The input is a RAM copy beginning at metadata offset 0x00. */
static unsigned long ReadLe32(const unsigned char *p)
{
    return (unsigned long)p[0] |
           ((unsigned long)p[1] << 8) |
           ((unsigned long)p[2] << 16) |
           ((unsigned long)p[3] << 24);
}

static unsigned long ExampleCan3AppStart(const unsigned char *metadata)
{
    return ReadLe32(metadata + 0x1C);
}

static unsigned long ExampleBur1AppStart(const unsigned char *metadata)
{
    return ReadLe32(metadata + 0x18);
}
```

此示例只示意LE與不同契約offset，不是完整驗證器。先驗magic、format、完整長度與CRC再使用欄位； 讀RL78遠端Flash須依實際編譯器的far pointer / 讀取API搬到RAM，不能將上例near pointer直接指向所有Flash。 Flash erase / write必須走該MCU的FSL / RFD及保護流程，不能用普通pointer賦值取代。

4. CAN payload逐byte格式

Byte0..Byte7指CAN Data payload，不含ID、DLC、CAN硬體CRC / ACK。不是每個command都DLC8。 下表的「-」代表沒有傳送該byte，不是要求補00。所有offset / size / CRC數值皆LE。

F13 CAN Upgrade：control request（ID0x700）

Command	DLC	B0	B1	B2	B3	B4	B5	B6	B7
HELLO	1	01	-	-	-	-	-	-	-
START	8	02	protocol	target0	target1	size0	size1	size2	size3
END	1	04	-	-	-	-	-	-	-
ABORT	1	05	-	-	-	-	-	-	-
ENTER	1	06	-	-	-	-	-	-	-
INFO	2	07	page	-	-	-	-	-	-
METADATA	8	08	metaOff0	metaOff1	00	data0	data1	data2	data3
SEAL	8	09	00	00	00	metaCrc0	metaCrc1	metaCrc2	metaCrc3
METADATA_RESET	1	0A	-	-	-	-	-	-	-
BLOCK_BEGIN	8	0B	imageOff0	imageOff1	imageOff2	blockCrc0	blockCrc1	blockCrc2	blockCrc3
BLOCK_COMMIT	8	0C	imageOff0	imageOff1	imageOff2	blockCrc0	blockCrc1	blockCrc2	blockCrc3

protocol目前=05；target目前=0xF13B，所以START B2 / B3=3B / F1。 App只處理INFO / ENTER；HELLO與更新命令供Boot使用，已在Boot時不需再送ENTER。 本表ID為目前F13範例；Tool仍須依metadata / live INFO核對。raw data base ID與block size由INFO page0x1A宣告， 不在metadata的request_id / response_id欄位內；移植時三組ID不得重疊。 size為整個BIN長度、不是capacity。metaOff相對256-byte metadata；imageOff相對整個BIN、64-byte對齊。 data0..3是本次METADATA chunk的4個原始byte；metaCrc是完整256-byte CRC，blockCrc是本區塊實際長度CRC。 BEGIN與COMMIT必須使用相同imageOff / blockCrc；BEGIN不回ACK，COMMIT驗證並寫入成功才回ACK。

Raw image frame（ID0x720..0x727）

Byte	意義
B0	BIN[imageOff + segment*8 + 0]
B1	BIN[imageOff + segment*8 + 1]
B2	BIN[imageOff + segment*8 + 2]
B3	BIN[imageOff + segment*8 + 3]
B4	BIN[imageOff + segment*8 + 4]
B5	BIN[imageOff + segment*8 + 5]
B6	BIN[imageOff + segment*8 + 6]
B7	BIN[imageOff + segment*8 + 7]

segment=CAN_ID-0x720，範圍0..7。這8個byte全部是image，不另放command / sequence / CRC8。 最後不足8但仍4-byte對齊時DLC4，只傳B0..B3；未使用byte不寫Flash。raw frame本身不逐筆回覆。

Response（ID0x701）

Response	DLC	B0	B1	B2	B3	B4	B5	B6	B7
HELLO	8	01	status	protocol	format	target0	target1	capKiB0	capKiB1
INFO	8	07	status	page	state	value0	value1	value2	value3
ENTER	2	06	status	-	-	-	-	-	-
common ACK / NACK	8	echoedCmd	status	detail0	detail1	value0	value1	value2	value3

status=00為成功，非零必須依command / 狀態解讀；例如01 BAD_COMMAND、03 BAD_SEQUENCE、 05 FLASH_ERROR、06 CRC_ERROR、09 BAD_IMAGE。INFO state=1 App、2 Boot；INFO page0的value包含target / protocol / format， 其他page與定義見CAN協定。

ACK command	detail（B2..3）	value（B4..7）
START	0	App起點絕對Flash address
METADATA_RESET	0	256
METADATA	下一個metadata offset	256

ACK command	detail (B2..3)	value (B4..7)
SEAL	256	256
BLOCK_COMMIT	(imageOff / 64)低16 bits	下一個預期Flash絕對address
END	(image_size / 4)低16 bits	device計算的完整image CRC32
ABORT	當前expected image byte offset / 4低16 bits	當前Flash address；不等於可斷線續傳承諾

只讀INFO範例：TX ID0x700 / DLC2 / `07 00`；Boot正常回ID0x701 / DLC8 / `07 00 00 02 3B F1 05 06`，分別是command、OK、page0、Boot state、target LE16、protocol、format。 第一個64-byte block成功的ACK為`0C 00 00 00 40 20 00 00`，最後四byte解為下一個Flash address 0x02040。 ACK範例不是完整燒錄流程；必須先完成live比對、metadata staging / SEAL及START，不能直接拿任意raw frame寫入。

與Custom Transfer / Bridge Self Test的區別

Byte	Transfer request 0x710	Transfer response 0x711	Self Test 0x123
B0	A5 magic	5A magic	11
B1	protocol 01	protocol 01	22
B2	非零sequence	對應sequence	33
B3	command	對應command；錯誤為7F	44
B4	argument0	response0	55
B5	argument1	response1	66
B6	argument2	response2	77
B7	CRC8(Byte0..6)	CRC8(Byte0..6)	counter，每次Start從00遞增、FF後回00

Transfer固定DLC8，CRC8 polynomial07、init00、xorout00、不反射；命令參數見 Transfer協定。 Self Test不是更新協定，也沒有Byte7 CRC；需CAN analyzer觀察實際bus訊號。 F24 host自身的Misc更新則是Host Link UART / BUR1， 不是將其UART payload硬套到上面的CAN八byte表。

5. 改成自己的專案：修改順序與驗證

1. 先決定角色與更新介面。F24 MCU CAN device不能直接拿F24 host的HOST_APP / BUR1 / Misc當CAN更新流程。
2. 固定MCU / Flash erase-write單位、Boot / App分區、完整向量 / startup / handoff RAM及IRQ ABI； 新MCU先對齊其clock、CAN driver與FSL / RFD，不能移植另一系列register或Flash library。

3. 建立自己的producer及validator。F13參考app_image.json→生成ftable.inc + 組語 + post-build與Boot config； F24 host參考layout header + app_image.json + packager與Bu_ParseManifest。依上方欄位 / 原始碼表逐項修改。
4. 同一MCU / 固定契約只改App功能、版本或一般ISR時，使用該App自己的JSON / mapping / 版本常數及封裝流程， 不重產共用Boot。改分區、role、更新協定、ABI或metadata格式則必須重新驗證並安裝matching Boot/App。
5. 讓linker / post-build決定image_size及CRC地址。BIN不含絕對load address；Tool必須由metadata及live INFO核對， 不能在UI改App Start就把已link好的image搬到其他位置。
6. 保留128-byte core + 128-byte reserved FF；日後啟用reserved欄位須明確定義format / policy版本， 同步producer、Boot、Tool與測試，不能只在App寫新值。
7. 先離線檢查所有256 metadata bytes、ASCII / LE / role / transport / geometry / IRQ ABI、BIN長度與CRC。 再做read-only live INFO比對，之後才進入metadata staging / SEAL / START / block / END。
8. 實板須另測錯映像、重送、斷線、斷電、不完整映像不跳App、reset後版本 / IRQ及Boot復原。 文件 / model測試不等於實體Flash或CAN bus驗證。

閱讀完整向量 / startup工程步驟：Boot/App移植指南。 G16新版I2C與Device UART流程不因本文件而開放；F24 CAN device的分區 / 韌體需另行開發及驗證。

文件維護

本文件FLASH_REFERENCE區塊同步至Bootloader web；ISP web及下載總覽連回同一入口。 逐byte範例由scripts/generate_flash_reference.py依兩份實際BIN產生，只改文件，不修改BIN / HEX / MOT。 產表工具只核對範例格式、長度、CRC及reserved，不代替Tool / Boot的完整安全驗證。 若更換分區或範例MCU，還須同步第1節Flash表、文字中的address例子與來源程式說明，不能只刷新數值表。 配套範例變更時明確刷新並檢查，不把表內範例值當成所有App的固定配置：

```
python scripts/generate_flash_reference.py `
  --f13-image ../RL78_F13_Boot_loader_CAN_app/DefaultBuild/RL78_F13_Boot_loader_CAN_app_update.bin `
  --f24-image firmware/RL78_F24_Bridge_app/DefaultBuild/RL78_F24_Bridge_app_bridge_update.bin
```

加--check只比較，不寫文件；再執行sync_flash_reference.ps1及-CheckOnly，更新本指南PDF。